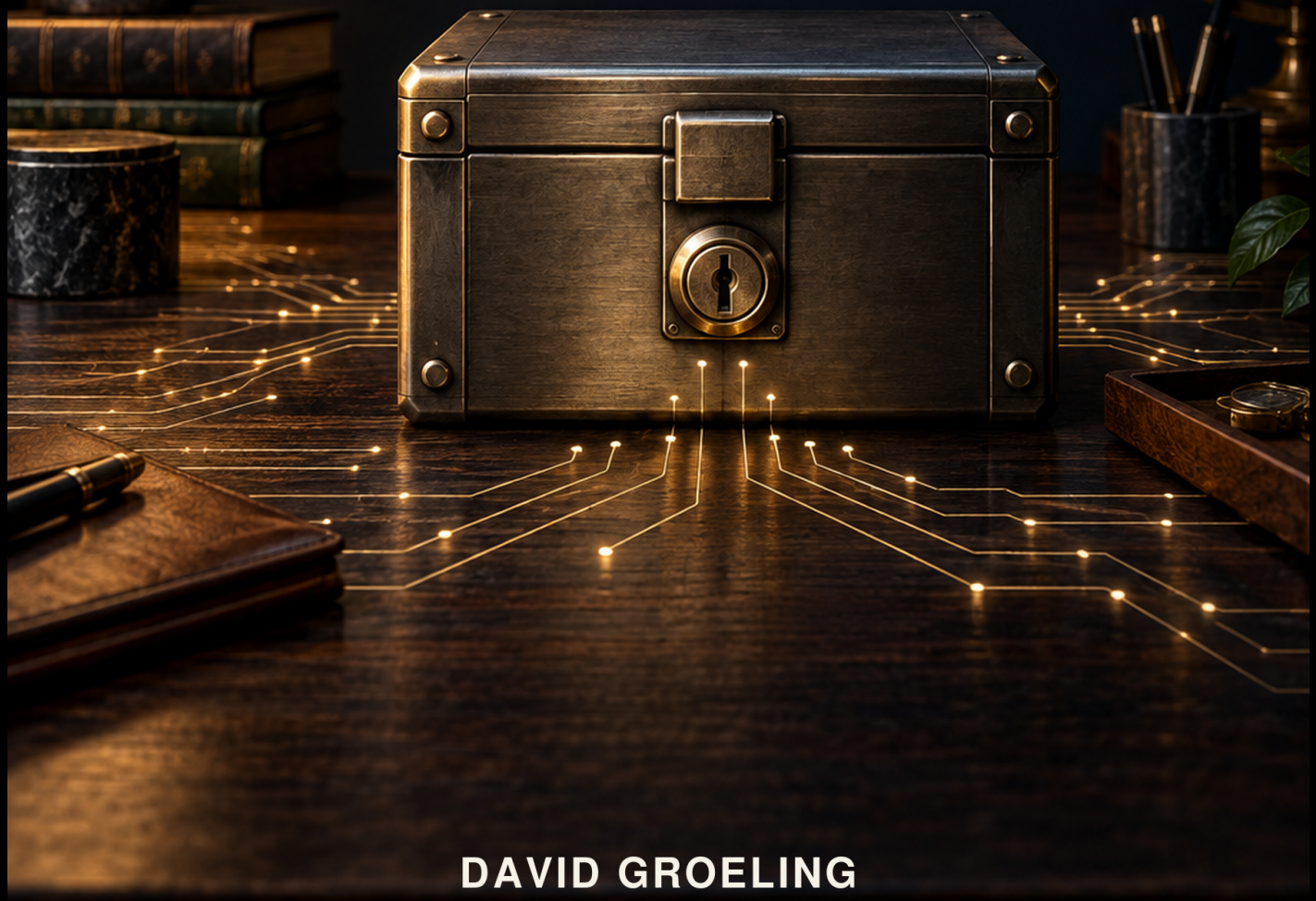
PERSONAL INFORMATION CONTROL

KEEPING OWNERSHIP OF YOUR CREDENTIALS, DATA, AND RECOVERY - ON YOUR OWN TERMS



DAVID GROELING

AJUMBLEDBOX

Table of Contents

Introduction **1**

A Baseline: The Disconnected Machine **2**

How We Connect, and How We're Exposed **2**

The Password Problem **2**

- Passwords 3

Is Anything Actually Secure? **3**

Authentication Methods Available Today **4**

The Memory Problem **4**

When Systems Fail **5**

What is Really Reasonable **5**

A Physical Method: The Handwritten Ledger **5**

The Credential Catch **6**

Generating Your Own Passwords **6**

- Script 1 - PowerShell 7.x 7
- Script 2 - PowerShell 5.x and 7.x 7
- Python 7
- OpenSSL 8

Where to Store What You Generate **9**

Keeping Control in Your Own Hands **10**

Methods **11**

- What Is the Difference Between HDD and SSD? 11
- VHD/VHDX and VeraCrypt Container Files 11

SDelete: What Is It and What Does It Do? **12**

Table of Contents - continued

What Is ReTrim and Why Does It Apply to SSD? **12**

Delete, Shift+Delete, Cut and Move 12

Files and Their Remnants **13**

Deleting Files Created Outside a Container 13

Deleting Files Created Inside a Container 13

Deleting a VHD/VHDX or VeraCrypt Container 13

Procedural Steps **14**

Creating a VeraCrypt Container File **16**

Walkthrough: Creating a VeraCrypt Container 16

Starting the Wizard 16

Volume Location 16

Encryption Options 17

Volume Size and Password 17

Filesystem Selection 17

Finishing Up 17

Aspects Not Yet Covered **18**

Backing Up Your Volume Header 18

Resizing a Container 18

What Actually Counts as an Air Gap **18**

Secure Deletion **18**

Personal Information Control

Keeping ownership of your credentials, data, and recovery — on your own terms

Authored Content

© 2013-2026 David Groeling / Ajumbledbox. All rights reserved.

Revision 8 — September 23, 2026

No implied or given Liability. Use this information at your own risk.

Introduction

The past year I have had several conversations in respect to consumer (everyday user) information on their own machines — desktop computers, laptop computers, notebooks, tablets, and phones — and their concerns about what is exposed.

In particular, there is an escalating level of what the user exposes to environments other than their own machine.

To fully understand this, let's take a look at a few things.

A Baseline: The Disconnected Machine

For the most part, this all deals with how we use our machines. If we use our machines only as they are — meaning they are not connected to anything, with no internet connection of any form (telephone lines, Wi-Fi, cable, Ethernet, Bluetooth, fiber, NFC), and whatever other avenues may exist for individual circumstances — exposure is not entirely eliminated, but it is nearly as close as anyone would expect. For all intents and purposes, you are considered not connected to anything.

But that is not how we use our machines.

How We Connect, and How We're Exposed

Let's focus on three types of machines, as they are more similar in nature than other forms of devices: desktops, laptops, and notebooks. Three major operating system families exist across these: Microsoft Windows based, Apple Inc. based, and Linux based.

We all, in general, connect to other avenues of communication via some sort of internet or wireless activity. When you use a web browser, you are connecting to another machine — or group of machines — in some other location. But you are also connecting through any number of locations and machines before you arrive at your final destination. All of them have access to the information passing between you and your intended destination. Then there are things that aren't part of the actual chain of connections at all, but are grabbing or monitoring that data as it passes — not interacting with you or your destination, just observing.

Everything you type and everything you see on your screen is, in that sense, in open communication. Your browser incorporates some attempt at encryption between you and the end location of your choice.

In most cases, web browsers are the direct avenue of connection, but things have evolved into application-bound connections as well. Most of that connection works similarly to your web browser, with added control points meant to provide some sort of confined communication.

The Password Problem

What this is focusing on is the sensitive information you look to keep out of the reach of others. Many additional tools, over the course of the time we have generally used these machines, have been updated, included, or provided as different methods in an attempt to convince you that all is good and well.

Passwords

Passwords come in many variations, including wildcards or special characters. The length of these things is called a string — a set of data you are meant to hold or enter. A password you are required to type in is a data set: bobby, or, Bobby, or, Bobby123, or, Bobby1$2#3!, or, Bobby1_2_3, or, Bobby 1 2 3. bobby was Bobby as is Bobb123.
All of those are a string, the same type of thing.

When we go to different places — Amazon, Google, Yahoo, Microsoft, Apple, PayPal, Etsy, Temu, Facebook, Instagram, or anywhere else — in most cases we are required to present these credentials if we are actually interacting with the site. Aka Login to the site.
In most cases, you can go anywhere and not be presented with a required access method just to see what they're about. But as soon as we start to actually interact with most things, that changes.

In many cases we are presented with entering a username and password. Some places automatically decide that only an email address is acceptable; others allow you to choose a username and include your email; others require a phone number.

Almost all require a password, its length and complexity can be different. Some require just a few characters; some state the password must be between a minimum and maximum length in the number of characters and may need to include or exclude wildcards and special characters. It is a massively confusing set of requirements to work through when trying to understand our own security — and for many of us, this is exactly the process we go through with banks and merchants today.

Is Anything Actually Secure?

What is secure, and what isn't, when all of these avenues of connection exist?

Several things exist in an attempt to provide the user with some sort of reassurance. As stated earlier, web browsers have attempted to update their secure connection methods over time. Then there are the individual applications — apps — which is really just a more modern name, and a more modern understanding of what used to be called a program or executable. Much of the things we use today are not installed on our own machines.

Restrictive measures are constantly being introduced to the connection avenues of these programs. Restricting the communication to you and your intended communication end point — a bank, a merchant, an email to a friend or business. Alongside this is the cryptographic aspect: encryption between you and your intended endpoint.

All of these things can interact with each other at the same time, but they are each vulnerable at different points.

TTo give a basic sense, apps, web browsers, web servers, application servers, and your cable or cell provider all play a part in what your computer, laptop, tablet, or mobile device is doing. Your role is providing the credentials needed to access and connect to these endpoints.

So what do we do with all of these connection points and the access credentials we are required to hold? Using the same name, email address, phone number, and passwords or phrases everywhere is a very bad idea — but many people do exactly that.

Authentication Methods Available Today

Out of the different attempts to help secure your information, several types of logins and checks have been implemented:

- Password managers
- Windows Hello
- TOTP (time-based one-time passcodes)
- Authenticator apps
- Hardware keys
- Passkeys
- Email notifications
- Push notifications to authenticated devices

There are some other forms as well, such as physical items and similar mechanisms.

At some point, much of this still boils down to the original criteria: names, usernames, email addresses, street addresses, phone numbers, secret words, phrases, and answers to security questions.

The Memory Problem

What can you do to help keep control, or some semblance of control? Memory is a limited information source. Few people have a long-lasting memory for complex or confusing information, and passwords alone are confusing enough as a single word. Then we have the overwhelming amount of these things we are to retain.

In a world where most people no longer write much of anything down — let alone maintain a vast catalog of personal information — if we did keep such a catalog, we would wind up with a long list of things we'd need to physically type on a keyboard or verbalize to another application.
Exposure is everywhere!

The “long form” information — passwords, passphrases, recovery phrases, recovery keys, email and physical addresses, answers to security questions, phone numbers, usernames, and so on.
Given its combined length it would be monumental in size and scope.

When Systems Fail

Then there is the risk of our own machines being compromised, or simply breaking down, leaving us locked out of our machines.

Life becomes far more complicated, even devastating, when information is lost — not so much the personal data itself. Anything that creates that kind of loss matters. Where is the individual's backup, and where is the access to it?

We all use keyboards and type to varying degrees of accuracy — some very good, some very bad, most somewhere in between. Typos, misspelled words, forgotten characters, missing spaces, bad keyboards — any number of things affect the accuracy of what we see and type. And that becomes a significant problem. When we enter our credentials via a keyboard. How does an individual secure themselves while still providing the ability to recover?

What is Really Reasonable

Many of the methods companies have attempted to provide, as listed above, hint at the same underlying message:
Someone else is doing it for you, so once again you're left in the hands of others — even while being told the responsibility is entirely yours. “ Do not share this with anyone”.
Yet that is exactly what you are doing directly from the start.

A methodology for handling this is not a simple thing to think through or understand, once you take in everything — and recognize how many parts aren't even stated outright, only hinted at. A great deal happens that we don't see or even know about. “Exposure” isn't quite the right word for it. In reality, it's a wide, gaping hole in this illusion of security.

The digital world has its flaws and limitations. Nothing is as simple as point-and-click when you're looking for real security.

A Physical Method: The Handwritten Ledger

Here is one method that can be applied, and is used: a hard, handwritten copy. Your source set of documents or a ledger.

As a daily working reference if you have more than just a few entries — and we all do. Now becomes a fragmented, often scattered list. But it is the source you maintain, and can keep out of reach of others, in both the digital world and the physical world. Where to keep such a thing is your own decision, but it shouldn't be in the desk drawer next to the machine you're using. Once again introducing a complicated place of interaction when you need to use it.

The credential catch

If you're not providing your own credentials, someone else is doing it for you. That isn't such a bad thing in many cases as we are all limited to varying degrees on how we think of these things. Many of the attempted points of help that our end point interactions are trying to implement. Are forcing you to not be so predictable/Entropy. For example the password 123456789 actually was and is still used. That fails 100% of the time to secure your access. This still exists and is listed on the worst password lists.
But if you're not having someone else decide for you what your password/passphrase should be. We on average have little clue on how to do that. Other than going to another source for that generation.
Most fail to recognize that the hardware is right at your finger tips. There is in many respects little difference to what you have and to what exists elsewhere. Just because something can do something quicker doesn't mean it isn't doing the same thing as the slower thing. And when we state quicker what does that really mean? Is a 3 second gap causing any undue stress as compared to 10 microseconds. The difference is discarded as if it never existed. You never even notice it.

Generating Your Own Passwords

When it comes to passwords — how and what to use — whether you use Google or some other program to create passwords through password generators or suggestions, it is something someone or something else is doing, not you, and a remnant of that generation and password is retained somewhere. You can generate your own passwords. And that is likely the better choice. You can decide everything on your own, though that becomes a bit of a problem when it comes to length and the genuinely non-random tendencies of a single individual choosing characters by hand.

Windows incorporates very strong cryptographic software directly. Alternatively, you could install a program designed for that purpose — OpenSSL is one avenue, and it is included with many pieces of software — but Windows itself is a very strong, good choice if you know what you can create with it.

Different variations of passwords can be generated via algorithms and methods. Using Windows PowerShell is the common tool today, installed on almost all active machines. Command Prompt still exists, but the terminal defaults to PowerShell in most cases.
Two current versions are today's standards: PowerShell 5.x and PowerShell 7.x.

If using on a boot drive Windows has system encryption's 64-byte pre-boot password limit.
Below scripts. PowerShell. The first uses PowerShell 7.x. The second runs correctly with PowerShell 5.x and PowerShell 7.x.

Script 1 — PowerShell 7.x

```
$chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()_+-='

$password = -join (1..32 | ForEach-Object {
    $chars[[System.Security.Cryptography.RandomNumberGenerator]::GetInt32($chars.Length)]
})

$password
```

Script 2 — PowerShell 5.x and 7.x

```
$chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()_+-='
$crypto = [System.Security.Cryptography.RandomNumberGenerator]::Create()
function Get-UnbiasedIndex {
        param([int]$UpperBound)
        $byte = New-Object byte[] 1
        $limit = 256 - (256 % $UpperBound)
        do {
        $crypto.GetBytes($byte)
        } while ([int]$byte[0] -ge $limit)
        [int]$byte[0] % $UpperBound
}
try {
        $password = -join (1..32 | ForEach-Object {
        $chars[(Get-UnbiasedIndex $chars.Length)]
        })
        $password}
finally {
        $crypto.Dispose()
}
```

Both of these scripts produce a 32-character password. To change the length, adjust the range 1..32 — for example, 1..8 for an 8-character password, or 1..10 for a 10-character password. Both avoid modulo-bias. Both are ~200 bits of Entropy

Python

If python is available this is also a good generator
python -c "import secrets; print(secrets.token_urlsafe(32))"
43 ASCII characters 256 bits of Entropy System Drive Safe Yes
Available Characters
ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz 0123456789 - _

OpenSSL

OpenSSL, like Python, offers a cross-platform alternative to these PowerShell scripts — useful on Windows, macOS or Linux, or anywhere OpenSSL is already installed.
Commands: Better choice is likely -hex 32 but that is up to the user Both produce 256 bits of Entropy
openssl rand -hex 32
generates **32 cryptographically secure random bytes** and displays them as hexadecimal text.

- `openssl` — runs OpenSSL.
- `rand` — generates cryptographically secure random data using OpenSSL's random-number generator.
- `-hex` — converts each random byte into two hexadecimal characters (`0–9` and `a–f`).
- `32` — requests **32 random bytes**.

The result is:

- **32 bytes**
- **256 bits** of randomness
- **64 hexadecimal characters**
- Usually followed by a newline
- Hexadecimal notation is a base-16 number system that uses sixteen distinct symbols (0–9 and a–f) to represent values all lower case, Can change to uppercase with this command
  openssl rand -hex 32 | tr 'a-f' 'A-F'

Example output:
89c93bd540ce33a0cd195734d12a3ce23e77a163f714e3dbdf01981ca32bfb87

openssl rand -base64 32
generates **32 cryptographically secure random bytes** and displays them as Base64 text.

- `openssl` — runs OpenSSL.
- `rand` — generates cryptographically secure random data.
- `-base64` — converts the random bytes into Base64 characters.
- `32` — requests **32 random bytes**, not 32 characters.

The result is:

- **32 random bytes**
- **256 bits of randomness**
- **44 Base64 characters**
- Usually **one `=` padding character** at the end, included within those 44 characters
- Followed by a newline when displayed

The characters can come from: A–Z a–z 0–9 + / and `=` can be used as padding.
Example: Note here you must include the padding character =
j8NE6TWj+I5GkV9Ihlg0ePR5g/c0GUv3zZ4r8g2jF2Y=

The Important point here is that the understanding of what is happening is clear.
The user is being asked to have various types of passwords/passphrases/Keys/Tokens. Different methods of logins to have access to what belongs to you.
These passwords/passphrases/Keys/Tokens can be created by you. Which include the same or similar randomness/Entropy that is being decided for you by others.
Using PowerShell or OpenSSL to generate your own elements is just one way to do that. There are variations that can be applied to both methods. That helps meet the standards set by others for the actual granting of access. Remember it is others who are in control of you actually gaining access. Their login or access criteria is controlled by them. Sometimes it's the length of a string, sometimes it's an inclusion of a different form of Character uppercase vs lowercase or plus those or inclusions of numbers and or wildcards or special characters. The big point here is.
Be random.
Do not use the same exact logins for everything.
A different login for everything is always a better choice.
Including multiple forms of verification before access is better.

> These are the aspects that someone has a user name and password but still needs a verifier to grant Access. An authenticator app and sms pin something else before access is granted.

PowerShell is just the software we are using to interact with what's native in Windows and or OpenSSL if installed.

If I have time I will provide working models at the end.

Where to Store What You Generate

Where do you store all of these items, out of reach of a hacked or compromised system?

You can keep most, if not all, applications from retaining your login information, but then you have to manually enter it each time you access those things. This is the inevitable result — whether we recognize it or not, we are doing this multiple times a day, some of us many more times than just a few.

Hence, this is exactly where and why all of these other avenues and attempts have been made to give you some sort of feeling of security. Password managers keep all of your information out on some server, somewhere in the world, only to be abused or misused eventually — likely exposed by a bad company or a poorly designed system.

Keeping Control in Your Own Hands

So where can you keep things on your own system, to make this information securely ready to use, without having it elsewhere?
You could create locked, encrypted files — but this leaves files that are still visible in your system's file history path. That gives anyone who compromises or locks you out of your system, with or without your knowledge, eventual knowledge to the file you were trying to protect. Access is still an issue but they are in control.

You could store this information on paper. Very complicated over time, though somewhat secure in your own residence.
You could store it on another part of your system, such as another drive or partition.
You could store it on another removable drive, such as a pen drive, USB drive, or backup drive.
You could store it on the web using paid cloud storage or free backups.
But unless encrypted, you're giving others direct access.
Maybe you depend on synced files, which again exposes secure and unsecured documents ready to be read.
Almost every option, except pen and paper or a disconnected drive, places your information in someone else's possession—which is a bad place to keep important things.

An alternative way. It may not be the absolute best way or a way that is convenient but it is a way nonetheless to gain some control of what is so important to you. Whether it meets with your desired level of containment is your choice. There are always better ways and there are always poor ways. Leaving everything to others in my opinion is a poor way.

One method is to use encrypted files but not just that, encrypted drives. and then the ability to attempt to erase or obscure their existence once they are no longer needed or being used. For example, we delete something on our machines. Is it really gone or destroyed like it never existed? That may or may not be true. We have memory chips, we have cache, we have file fragments and the list goes on.
This is just an attempt to mitigate some of those areas.

This is the plan.

Virtual drives, containers, files. Meet's the convenience.
Encrypted with modern cryptography. Meet's the security.
Stores Master Ledger. Meet's retention.
File, Containers and VHD/VHDX (Virtual Hard Drives) removals. Meet's security
Clearing of those shadow elements left over. Supports security.
Portability to ensure off machine control and or backup. Meet's security and retention.

Methods

We will give an example of two methods
Bitlocker and VeraCrypt
Bitlocker is a native tooling included in Windows other than Home versions.
When we refer to Bitlocker we refer to the full version. I state that because Windows Home versions do use the same technology as Bitlocker but it is not labeled as such and it is a limited set of options to it. Which is why it's called Device Encryption. Also other toolings sdelete. And ReTrim functions of a SSD, VHD/VHDX.

Putting all those things together gets us to a methodology to try and actually mitigate this growing list of things we are trying to shield access to that belong to us.

Some basics understandings:

This Setup is not something that you need to do often no more than once hopefully a pattern is established that provides you with a one time setup and follow procedure
VeraCrypt and Windows native cryptographic tooling. We included OpenSSL/Python to give understanding that other tools common to users are readily available when it comes to creating passwords. And to be clear the online password generators aren't necessarily bad in and of themselves; it's the trail you leave that is the issue.
Two main parts actually exist. Beyond VeraCrypt and Bitlocker. Platter hard drives and solid state hard drives referred to respectively as HDD and SSD.

What is the difference between HDD and SSD?

An HDD (hard disk drive) stores data on spinning magnetic platters and uses a mechanical read/write head to access that data, while an SSD (solid-state drive) stores data electronically in flash memory chips with no moving parts.

What is the difference between VHD/VHDX and VeraCrypt container file?

A VHD/VHDX is a Microsoft virtual hard disk format that stores a raw disk image in a single file and can be mounted by the operating system as a normal drive without any special software. A VeraCrypt container is an encrypted file that VeraCrypt mounts as a virtual disk, with the entire contents protected by strong encryption and authentication. The key difference is that VHD/VHDX is a general-purpose virtual disk format with no built-in encryption, while a VeraCrypt container is specifically an encrypted volume format that requires VeraCrypt to access.

Sdelete: What is it and what does it do?

SDelete (Secure Delete) is a free command-line utility from Microsoft's Sysinternals suite that permanently removes files and cleans free disk space by overwriting the actual data on the drive. Unlike a normal delete, which only marks space as available, SDelete writes random or patterned data over the file's clusters (and optionally over free space) so recovery tools cannot reconstruct the original content. It is commonly used to securely erase sensitive files and to sanitize previously deleted data on NTFS volumes.
Though it works on non NTFS volumes it may be limited compared to NTFS volumes. If this is a concern there are other alternatives to do exactly what this description defines on those other Formats.

What is ReTrim and why doesn't it apply to HDD only SSD?

ReTrim is a Windows maintenance operation that re-sends TRIM (or UNMAP) commands to an SSD to tell it which blocks are no longer in use so the drive can erase and reuse them. It exists because TRIM requests are processed asynchronously and can be dropped under heavy load, so ReTrim ensures all currently free space is properly reported to the SSD controller. It only applies to SSDs (and similar flash/thin-provisioned storage) because HDDs don't use TRIM or garbage collection in the same way; they rely on mechanical rewrites and benefit from defragmentation instead of block-level trim operations.

* Note here: Sdelete is used to Clear remnants on an HDD and ReTrim we will try to do the same on SSD.
* Note here: Secure erase functions / Disposal. Are the preferred permanent choices.

Right click delete, vs highlight + del key, vs shift key + del key?

Right-click → Delete and pressing the Delete key after selecting a file do the same thing in Windows: they move the file to the Recycle Bin so it can be restored later. Holding Shift while pressing Delete (Shift+Delete) bypasses the Recycle Bin and permanently deletes the file immediately, subject only to your confirmation prompt and Recycle Bin settings. In all three cases the data remains on disk until overwritten, but only Shift+Delete skips the Recycle Bin step and cannot be undone via "Restore" from there.

Cut and paste from one directory location to another vs highlight and drag from one directory to another?

Cut + Paste always performs a move operation: the file is removed from the source directory and placed in the destination, with no duplicate left behind. Dragging files with the mouse can either move or copy depending on context: on the same drive it usually moves, but across different drives or to network locations it usually copies unless you hold modifier keys. So Cut + Paste is explicit and consistent, while drag-and-drop behavior changes based on source/destination and any Ctrl/Shift keys you hold.
* Note here: This to show that there are file remnants left and our purpose of sdelete and retrim.

Files and their remnants when created outside the VHD/VHDX and or VeraCrypt.

A file created outside a VHD/VHDX or VeraCrypt container, or created elsewhere and then copied or moved into one, leaves traces behind. Deleting it does not remove them. The file's data stays on the drive until something overwrites it, and Windows can hold other copies, such as restore snapshots and the page file. Moving a file into a container only adds an encrypted copy. The original's remnants stay where it was.
The purpose is to leave as little behind as possible, so that no one can reconstruct what you were trying to protect. The most reliable approach is to create the file inside the container from the start, so its contents are encrypted as they are first written.
*Note here: Repeat: This to show that there are file remnants left and our purpose of sdelete and retrim.

Procedure for Deleting and cleaning up a file created outside of VeraCrypt Container.

1) We delete the file with sdelete
   <code>
   sdelete64 -p 1 "C:\path\file1.txt"
   <code>
   *note: -p 1 is the number of passes. 1 is enough but you can use more why i dont know.
2) Now we need to clean up the volume drive/partition
   <code>
   sdelete64 -c C:
   <code>
   *note: -z is to zero all the free space on the drive/partition -c is to clean up free space
   *note: This is a confusing aspect as it is unclear in its definition.

* Note here: Step 1 is the same pattern used for when a file is created within the VHD/VHDX / VeraCrypt file.

Procedure for Deleting and cleaning up a file created Inside of VeraCrypt Container.

You can use the method above as noted. Or it is possible to just do this for the entire VHD/VHDX / VeraCrypt file.
Because it's already encrypted, unmounting it and just treating it as a file should be sufficient.

What to do with the VHD/VHDX / VeraCrypt container file when created?

The file, once created, is essentially a Virtual Hard Drive/VeraCrypt container file. Like all hard drives, it must be installed or mounted to interact with it. When dealing with a VeraCrypt container file, this requires an additional step. On Windows and macOS, the VeraCrypt application is used to mount the file. Because it is an encrypted file that acts as a VHD, it always remains encrypted. You must supply the password through VeraCrypt to open and mount the container file.
Open VeraCrypt app – Choose drive letter – Select VeraCrypt file – Press Mount button – Mount / Supply password
* Note here: This is not where to store it; this is about interacting with it.

Procedural Steps:

1) Download VeraCrypt, If you know how to verify the integrity of the file. (PGP key)
   https://veracrypt.io/en/Home.html
2) Download Microsoft Sysinternals Sdelete
   https://learn.microsoft.com/en-us/sysinternals/downloads/sdelete
3) Create the VeraCrypt Container file.
   a) Follow the procedure for this.
   b) Store the password to this container file on paper – your master paper backup ledger.
4) Do not close VeraCrypt yet. You will need it to proceed.
5) Mount the VeraCrypt Container file. From the VeraCrypt application mount button.
   a) Follow the procedure for this.
   b) Make sure your container works and is accessible; this will verify your password is correct.
   c) If there are no warnings or errors you will have a new drive listed.
   d) Open/Enter the drive just like any other drive or partition to verify its functionality.
6) Unmount the VeraCrypt container file from the VeraCrypt application using the unmount button.
7) Included at the bottom look for the Hash tool or use another method to hash a file.
   a) Tool AJFI - Ajumbledbox File Integrity. Will also include python 2 file hash compare alternative.
   b) Whichever method you use there are two methods you could use. Depending on where the container file exists and where you intend to keep it. And for a backup. For example you could have created the VeraCrypt container file on a drive installed in your system. But your intent is not to use it there at all. You could have done that but intend to move the file over to a removable drive aka USB connected drive. To verify that wherever you move the container file. We refer to hash values. Just like confirming the integrity of any file we use hash here in the same way.
   c) Hash the VeraCrypt container file before you do anything else with it. Do not use it when doing so.
   d) Copy the file to its intended location. Validate the hash by hashing the copied container file. If the two hash values are identical it is considered to be identical. Both tools allow for hashing 2 files at the same time. But you could choose to hash one file at a time. For that you can use AJFI and your own method of hashing a file. The included python script will not hash one file.
   e) Below is windows terminal Powershell commands to hash a file if you choose that to hash.
      i) Hash the original file >Get-FileHash "D:\MyContainer.hc" -Algorithm SHA256
      ii) Hash the copied file >Get-FileHash "E:\MyContainer.hc" -Algorithm SHA256
   f) Compare hash values they need to be identical
   g) Validate the function of the file by mounting the file from the VeraCrypt application.
   h) Create a test file in the container. Close drive then unmount drive
   i) Repeat process mount drive – enter password – open drive – validate contents of test file or files.
   j) If all is successful this is considered a working container file.
8) We assume you know what files you want to move into this container file.
   a) Copy all files to the open container
   b) Hash all original files
   c) Hash all copied files
   d) Validate the hash values
   e) Open all copied files if you can. If not as long as the Hash values are identical in theoretical practice they should all be functional.
9) Close the container and unmount it. Remount and open the container to validate the hash values one more time.
   a) If All hash values match move on to step the next step

10) Most of this setup is meant to protect identity credentials, not general content, though you may still want to secure important files. If credentials were the only concern, a small container would suffice. Some items, like a will or proprietary documents, may justify a larger container. Keeping a physical master copy stored securely and separately is critical. The overall goal remains the same: protecting what is on your machines from unauthorized access.
11) Sdelete original files
    a) `-p 1` means one overwrite pass. More passes do not provide a meaningful practical benefit on modern hard drives and add time.
    b) `-s` includes subfolders.
    c) SDelete also renames the file before deleting it to reduce filename remnants; Microsoft says it securely deletes file data, although it specifically notes that it does not remove file names already located in free disk space.
    d) In their original locations
        i) sdelete.exe -p 1 "C:\Path\To\Original\File.pdf"
    e) If what your trying to delete securely is a directory
        i) sdelete.exe -p 1 -s "C:\Path\To\OriginalFolder"
    f) If you created the VeraCrypt container file first on your installed drive and don't want to retain it. As your purpose is to use the copied container file on the removable drive. You would also use sdelete on the Container file. Just like d) i) above as it is a file.
12) The next step is to try and delete or remove what exists on the installed drive.
    a) AKA the drive on your computer. After you have removed your VeraCrypt Container file. Or decided to keep it on your installed drive in your computer case.
    b) Whatever choice you choose, it's time to clean up the installed drive in your computer case.
    c) For drive C we do Prompt:> sdelete64.exe -c C: (Or) >sdelete64.exe -c C:\
    d) The -z option seems to be the best choice for within VHD/VHDX drives.
    e) If a VeraCrypt container file when it is mounted it acts like a drive.
        i) Sdelete can be used on a Drive. sdelete64.exe -c C:\.
        ii) VeraCrypt container file mounted on L:\
        iii) Do not use sdelete on this type of volume sdelete64.exe -c L:\
        iv) When its time its best to use sdelete to actually delete the entire VeraCrypt container file/
    f) The correct or preferred method is to sdelete the VeraCrypt Container file completely.

This should complete the pattern. From this point on you should create the files you want to secure directly in the VeraCrypt container file. This should limit or avoid any remnants on the working machine.

Creating a VeraCrypt Container File / The Virtual Hard Drive VHD/VHDX

You can create a file container, or VHD, that is encrypted. There are different methods of doing this. VeraCrypt can do this. There are different methods to create a VHD/VHDX. To turn those into encrypted drives we could use Bitlocker. The distinction here is that just creating a VHD/VHDX doesn't make it secure. It's the encryption of that drive that does so. Where VeraCrypt is always in an encrypted state.

Once encrypted everything inside this type of container is encrypted. A more secure means of deleting its contents is also provided. If someone compromises your system — even through ransomware — they can't simply open what's inside this container.

- Two distinctions:
    - Software on our machines.
    - Things that belong to us that are not software. Files: Images, Documents, Data…

This distinction matters. Because restoring our machines and systems is one thing. Restoring our information is another. That is what actually needs to be secured.
The machine, system, or software itself is replaceable. It might be a bit of a pain to replace, but it is replaceable. Our data and items likely are not or with great difficulty.

VeraCrypt can create the container file. VeraCrypt container files are encrypted at all times closed or open. VHD/VHDX needs to be created with other software. VeraCrypt also encrypted its container file. Creating a VHD/VHDX is not encrypted on creation. BitLocker would be used to encrypt the VHD/VHDX

T**Walkthrough: Creating a VeraCrypt Container**

Starting the Wizard

Open VeraCrypt and select the Volumes tab. Click Create Volume. This opens the VeraCrypt Volume Creation Wizard. Select Create an encrypted file container, then Standard VeraCrypt volume.

Volume Location

You'll see a text box and a Select File button. These aren't two different choices — they're two ways of doing the same thing. You can type a full path and filename directly into the text box (for example, Z:\Baby), or click Select File and browse to a location. Either way, you're specifying the same thing: where the container file will live and what it will be called. If the file you type or select does not already exist, VeraCrypt will create it.
* Note here: 1) Do not point this at a file you want to keep. It will be deleted and a new VeraCrypt File will be created. 2) This does not encrypt an existing file. This creates an encrypted container file.

Encryption Options

Two different selection areas here at the top and the bottom.

- AES, selected by default, is a solid choice for most circumstances.
    - Other choices are yours to discover
- The hash algorithm/KDF choice, shown simply as “Argon2,”. In the Selector Dropdown menu.
    - This is a modern memory-hardened choice.

These are the two choices we choose. We suggest that you discover the best choice for your use.

Volume Size and Password

- Next you'll choose how large the container should be.
    - If you create two small its not the end of the world.
        - You can create more containers or Create a larger container and move items in there.
- Then you'll set the password that protects it.
    - You can use one of the password-generation scripts described earlier.
    - Or you can choose to create your own or Use other sources for passwords.
- You'll also see the option to require a keyfile.
    - If you enable this, mounting the container afterward requires both the password and the keyfile together — neither one alone will open the VeraCrypt container file.
    - This adds a second, physically separable requirement to opening the container, but also means losing the keyfile locks you out just as losing the password would.
- Below that is the PIM (Personal Iterations Multiplier) option.
    - Explore what a custom PIM value actually changes before setting one — a low PIM can weaken protection, and a high PIM slows down every future mount. For most people, leaving both the keyfile and PIM fields empty is the right call.

Filesystem Selection

- On the next screen, most Windows users end up choosing NTFS.
- FAT is actually what the wizard pre-selects by default.
    - Chosen for cross-platform compatibility, since a FAT container can be read on Windows, macOS, and Linux alike.
- FAT has a limit for file sizes of 4GB.
- For Windows if you are going to use files larger than 4GB. NTFS is the right choice instead.
- Or if you want to use VeraCrypt's Dynamic (sparse-file) container option, which only works on NTFS.

Finishing Up

- Follow the on-screen instructions for the rest of the wizard.
- Moving the mouse randomly to generate entropy
- Then clicking Format. Once that is completed, the container is ready to mount.

Aspects not yet covered
Backing Up Your Volume Header
Resizing a Container

What Actually Counts as an Air Gap

- Your computer has two hard drives installed
    - It doesnt matter if one is platter and one is nvme(solid state)
- They are both accessible at all times.
- You have the same computer with two hard drives installed. And you have one USB drive.
    - That USB drive is only active when connected to your computer when plugged in.
    - When that USB drive is not plugged in. There is no connection. That is the Air Gap
- If your computer breaks or is compromised your Air Gapped USB drive is unaffected.

Secure Deletion

Deleting files securely, or cleaning up what's left on the drive afterward, comes down to either retrim or SDelete Or some other option you are aware of. And which one is correct depends on the type of drive and on which stage of deletion you're addressing.

Step 11 of **Procedural Steps:**

Gives an understanding of how to accomplish this.

- SDelete is not the only file-overwrite option, and it is worth checking what is already on your system before installing anything new.
- Git if installed includes shred, from GNU coreutils, ships with Git for Windows and is reachable from its bundled Bash terminal with no separate install — a real, verified alternative already present on many Windows machines that have Git installed for other reasons. Its syntax overwrites a specific file's contents in place before removing it: shred -u -z -n <passes> <file>, where -u unlinks (deletes) the file afterward and -z adds a final pass of zeros to mask the shredding itself. On the same platter-versus-flash logic covered above, it is just as reliable as SDelete on an HDD and subject to the same wear-leveling limitation on an SSD.
- Whether TRIM is actually active on a drive can be checked directly: fsutil.
- Prompt:>fsutil behavior query DisableDeleteNotify
    - DisableDeleteNotify reports 0 when enabled, which is the Windows default for internal SATA and NVMe **SSDs.**

Windows also runs a weekly scheduled Optimize Drives task that re-issues TRIM for anything missed, on top of forcing it immediately with Optimize-Volume. Trim does not usually work on USB drives. It can but it is dependent on the hardware of the enclosure. What is not gone into detail is what about permanently getting rid of your actual hardware. The method is using secure-erase software and disposal of the drive if possible through destruction. For decommissioning an entire SSD rather than a single file or container, the reliable method is a firmware-level command — ATA Secure Erase.
What follows is Bitlocker